

Final Year Project Report

Group No. 08
Widget World
BS Computer Science, Batch 2020F

Supervised by: Miss. Sobia Ashar
Designation: Junior Lecturer
Co-Supervised by: Dr. Amir Mehmood
Designation: Associate Professor
SSUET

Submitted by:

2020F-CS-030	Obaid Hashmi
2020F-CS-038	Hammad Shoaib
2020F-CS-044	M Talha Tahir
2020F-CS-265	Khubaib Jamal



Department of Computer Science & Information Technology
Sir Syed University of Engineering & Technology
University Road, Karachi 75300

<http://www.ssuet.edu.pk>

Final Year Project Report

**Group No. 08
Widget World
BS COMPUTER SCIENCE
BATCH 2020F**

**Supervised by: Miss. Sobia Ashar
Designation: Junior Lecturer
Co-Supervised by: Dr. Amir Mehmood
Designation: Associate Professor
SSUET**

Submitted by

2020F-BCS-030	Obaid Hashmi
2020F-BCS-038	Hammad Shoaib
2020F-BCS-044	M Talha Tahir
2020F-BCS-265	Khubaib Jamal

**In Partial Fulfillment
Of the Requirements for the degree
Bachelor of Science in Computer Science**

**Department of Computer Science & Information Technology
Sir Syed University of Engineering & Technology
University Road, Karachi 75300
<http://www.ssu.edu.pk>**

August 2024

DECLARATION

We hereby declare that this project report entitled **“Widget World”** submitted to the **“Department of Computer Science and Information Technology”**, is a record of an original work done by us under the guidance of Supervisor **“Ms. Sobia Asher”** and Co-Supervisor **“Dr. Amir Mehmood”** and that no part has been plagiarized without citations. Also, this project work is submitted in the partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

Name: Obaid Hashmi

Signature: _____

Name: Hammad Shoaib

Signature: _____

Name: M. Talha Tahir

Signature: _____

Name: Khubaib Jamal

Signature: _____

Supervisor:

Miss Sobia Ashar

Junior Lecturer

Signature:

Co-Supervisor:

Dr. Amir Mehmood

Associate Professor

Date

ACKNOWLEDGMENTS

First and foremost, I would like to thank our Almighty Allah, for providing me the strength, courage of conviction and the strong sense of dedication to my project that enabled me to take it to the level that it has achieved today. Undoubtedly, I wouldn't have been now here, if it weren't for His Mercy and Blessings.

This Work would have been impossible without the copious amounts of help, patience, and encouragement from my kind supervisor “**Ms. Sobia Ashar**”. I would like to thank him for teaching me so much and for inspiring me with the example of hard work, providing great help and key information throughout the time required for the completion of this project report. I would also very special thanks to “**Dr. Amir Mehmood**” for his valuable support in the risk and cost estimation and in testing phase of this project. I would also like to thanks the entire faculty members for their able support and encouragement which enable me able to complete my project report.

To my family, thank you for letting me do what I want to do, supporting my decisions as crazy as they may be. You are always there for me. My deepest love and gratitude goes to my father and mother who played such a vital role all through my life by always placing my interests ahead of theirs. This project work is dedicated to my parents, thank you for seeing me through every step of the way, for praying for me and with me during times of trouble, and for rejoicing with me over every little triumph. During my studies my family constantly provided me the hope that I needed to complete my higher education. Also, thanks to all other member of my family and friends whose love and prayers are with me all the time.

COPYRIGHT

The copyright of this report belongs to the author under the terms of the copyright Act 1987 as qualified by Intellectual Property Policy of Sir Syed University of Engineering and Technology. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

© Batch 2020F, Department of Computer Science & Information Technology.

All right reserved.

ABSTRACT

This Final Year Project (FYP) focuses on accelerating mobile application development through the strategic use of Flutter widgets. Our primary objective is to offer cost-effective alternatives to current solutions by developing a web application that mirrors the functionalities of existing platforms while introducing innovative features to enhance the user experience. Utilizing Flutter, a flexible and powerful UI framework, we aim to create a user-friendly, responsive, and visually appealing web application. This solution is designed to streamline the development process, ensuring rapid and efficient creation of mobile applications. By providing a free, open-source platform, our project eliminates the need for costly subscriptions, thereby reducing development expenses. Our commitment to empowering developers with a robust toolset will contribute significantly to the broader community's mobile development capabilities, aligning with the ever-evolving demands of the digital landscape.

Table of Content

Chapter 1.....	12.
1. INTRODUCTION.....	12.
1.1 Background of study.....	12.
1.2 Problem Statement.....	12.
1.3 Objectives.....	13.
1.4 Scope of Study.....	13.
Chapter 2.....	15.
2. LITERATURE REVIEW.....	15.
2.1 Existing System.....	15.
2.1.1 Flutter Brick.....	15.
2.1.2 Flutter UI Design.....	15.
2.1.3 Flutter Gallery.....	16.
2.2 Comparative Analysis.....	17.
Chapter 3.....	18.
3. REQUIREMENT SPECIFICATIONS.....	18.
3.1 External Interface Requirement.....	18.
3.1.1 User Interface.....	18.
3.1.2 Software Interface.....	18.
3.1.3 Hardware Interface.....	18.
3.1.4 Communication Interface.....	19.
3.2 Functional Requirement.....	19.
3.3 Other Nonfunctional Requirement.....	20.
3.3.1 Performance Requirement.....	20.
3.3.2 Safety Requirement.....	20.
3.3.3 Security Requirement.....	20.
3.3.4 Software Quality Attributes.....	21.

3.4	Cost Estimation.....	21.
Chapter 4.....		22.
4.	SYSTEM DESIGN.....	22.
4.1	System Architecture Diagram.....	22.
4.2	High level Design: System Operation.....	23.
4.3	High Level Design: System Model.....	23.
4.4	Object Model.....	24.
4.5	Data Model.....	24.
4.6	GUI Design.....	25.
Chapter 5.....		26.
5.	BUISNESS MODEL.....	26.
5.1	Business Model Canvas.....	26.
5.1.1	Key Partners.....	26.
5.1.2	Key Activities.....	26.
5.1.3	Key Resources.....	27.
5.1.4	Value Proposition.....	27.
5.1.5	Customer Relationship.....	27.
5.1.6	Channel.....	28.
5.1.7	Cost Structure.....	28.
5.1.8	Revenue Stream.....	28.
5.1.9	Customer Segment.....	28.
Chapter 6.....		30.
6.	SYSTEM DEVELOPMENT METHODOLOGY.....	30.
6.1	Development Methodology.....	30.
6.2	Key Milestone.....	32.
6.3	Project Plan with Grant Chart.....	33.
6.4	Tools Used for Development.....	34.

6.5	System Implementation.....	34.
Chapter 7.....		37.
7.	SYSTEM TESTING AND EVALUATION.....	37.
7.1	Introduction.....	37.
7.1.1	Purpose.....	37
7.1.2	Objectives.....	37.
7.2	Test Methodology.....	37.
7.3	Test Plan.....	37.
7.4	Test Approach.....	38.
7.5	System Environment.....	38.
7.6	Test Entrance Criteria.....	38.
7.7	Test Acceptance Criteria.....	38.
7.8	Test Cases.....	39.
7.9	Bug Reporting.....	40.
7.9.1	Bug Tracking.....	40.
7.9.2	Bug Security Definitions.....	40.
7.9.3	Bug Priority Definitions.....	41.
7.9.4	Bug Life Cycle.....	41.
7.9.5	Bug Reports.....	43.
7.10	User Acceptance Testing.....	44.
Chapter 8.....		45.
8.	RESULT AND DISCUSSION.....	45.
8.1	Result.....	45.
8.2	Discussion.....	46.
Chapter 9.....		47.
9.	FUTURE WORK AND CONCLUSION.....	47.
9.1	Conclusion.....	47.

9.2 Future work.....	47.
REFERENCE:	49.
Appendices.....	50.
Appendix 1(User Manual)	50.
Appendix 2(Wall Poster)	55.
Appendix 3(Brochure).....	56.
Appendix 4(Standee).....	57.
Appendix 5 (Industry Collaboration Letter)	58.
Page For Remarks/Comments.....	59.
Page For Remarks/Comments.....	60.

List of Figures

4.1 System Architecture Diagram.....	22.
4.2 System Operations.....	23.
4.3 System Model.....	23.
4.4 Object Model.....	24.
4.5 Data Model.....	24.
4.6 User Interface Design.....	25.
6.1 Development Methodology.....	31.
6.2 Project Plan with Gantt Chart.....	33.
7.1 Bug Life Cycle.....	41.
8.1 Registration Screen.....	45.
8.2 Login Screen.....	45.
8.3 Publishing Widget.....	46.
8.4 User Dashboard.....	46.
8.5 Updating Widget.....	47.
8.6 Admin Dashboard.....	48.
8.7 Templates.....	49.

List of Tables

2.1 Comparative Analysis.....	17.
3.1 Functional Requirements.....	19.
3.2 Cost Estimation.....	21.
6.1 Key Milestone.....	32.
7.1 Test Plan.....	37.
7.2 Test Cases.....	39.
7.3 Bug Severity Definitions.....	40.
7.4 Bug Priority Definitions.....	41.
7.5 Bug Reports.....	43.
7.6 User Review # 1 (UAT)	46.
7.7 User Review # 2 (UAT)	46.
7.8 User Review # 3 (UAT)	46.

Chapter 1

Introduction

1.1 Project Background

This Final Year Project (FYP) aims to accelerate mobile application development through the utilization of Flutter widgets, with a primary focus on delivering cost-effective alternatives to current solutions. The project aims to develop a web app with functionalities comparable to existing websites while incorporating innovative features to enhance the user experience.

Our project strives to streamline the development process, ensuring rapid and efficient creation of mobile applications. By harnessing the power of Flutter, a versatile UI framework, we aim to build a user-friendly, responsive, and visually appealing web app. This FYP will not only save development time but also reduce expenses, as we will provide a free, open-source solution that eliminates the need for costly subscriptions.

Through this project, we aspire to empower developers with a powerful toolset, ultimately contributing to the broader community's mobile development capabilities. Our commitment to enhancing mobile app development aligns with the ever-evolving demands of the digital landscape.

1.2 Project Objectives

- Create a user-friendly platform housing various widgets that users can easily browse, select, and integrate into their projects.
- Enable users to seamlessly copy selected widgets into their project codebase, reducing development time and errors.
- Allow users to modify and personalize widgets to suit their project requirements, fostering adaptability and flexibility.
- Prioritize a smooth and intuitive interface for users to navigate, search, and select widgets, enhancing the overall experience.
- Foster a community-driven platform allowing users to contribute new widgets, share feedback, and collectively improve the repository.
- Provide comprehensive documentation for each widget, including usage guidelines.
- Implement a system to manage widget versions and ensure compatibility with different programming languages and frameworks.
- Offer templates to facilitate the construction of complete mobile applications, further expediting development processes and enhancing productivity.

- Develop a new Flutter widget library that provides a wider range of features and options than the existing library.
- Make it easier and faster for developers to create complex and interactive UIs using the new library.
- Improve the user experience of Flutter apps by making them more engaging and interactive.
- Give developers a competitive advantage by providing them with access to a powerful and versatile UI widget library.

1.3 Problem Statement

Flutter is a popular cross-platform mobile development framework that allows developers to create beautiful and engaging user interfaces for a variety of devices. However, the current set of Flutter widgets is limited in terms of its ability to create complex and interactive UIs. This can lead to longer development times and less efficient code.

To address this challenge, we propose the development of a new Flutter widget library that empowers developers to create rich and interactive UIs with greater ease and efficiency. The library will focus on providing advanced features such as customizable animations, interactive elements, responsive design, and accessibility support, enabling developers to craft truly immersive and intuitive user experiences.

1.4 Product Scope

The product, Widget World, aims to revolutionize the way developers approach UI development, particularly in the context of Flutter, by providing a comprehensive and user-friendly platform for accessing, customizing, and integrating widgets. The product will accomplish the following:

1. Comprehensive Widget Repository:

- **Broad Range of Widgets:** Provide a wide variety of widgets that cater to different functional areas, such as data visualization, navigation, forms, and user interaction.
- **Categorized and Searchable:** Offer an easily navigable and searchable repository where widgets are organized into categories based on their functionality and use-case.

2. Easy Widget Integration:

- **Copy and Paste Integration:** Allow developers to seamlessly copy and paste widget code into their projects, reducing development time and errors.
- **Compatibility with Flutter:** Ensure that all widgets are compatible with different Flutter versions, allowing smooth integration into existing projects.

3. Customization and Personalization:

- **Widget Customization:** Enable users to modify and personalize widgets according to their project requirements, including styling, functionality, and behavior.
- **Documentation and Examples:** Provide comprehensive usage guidelines, code snippets, and examples for each widget to guide users in customization.

4. Community-Driven Platform:

- **User Contributions:** Allow users to submit new widgets, share feedback, and participate in community discussions to improve the repository.
- **Feedback and Reviews:** Facilitate user reviews and ratings for widgets to ensure quality and foster community engagement.

5. Widget Version Management:

- **Versioning System:** Implement a system to manage widget versions and ensure compatibility across different programming languages and frameworks.

6. Advanced Pre-build Widget:

- **Innovative Widgets:** Develop a new Flutter widget library with a wide range of advanced and interactive UI components to enhance the developer experience.
- **Performance Optimization:** Focus on optimizing widgets for performance and usability to ensure smooth and efficient use in projects.

7. Improved User Experience:

- **Engaging UIs:** Enable developers to create complex and engaging UIs using the versatile widget library, improving the overall user experience of their apps.
- **Competitive Advantage:** Provide developers with access to a powerful UI widget library, giving them a competitive edge in the market.

8. Continuous Improvement and Support:

- **Ongoing Updates:** Regularly introduce new widgets, features, and improvements based on user feedback and industry trends.
- **Support Channels:** Offer support for developers, including forums, documentation, and direct assistance, to ensure a positive user experience.

By delivering on these key areas, Widget World will provide a comprehensive and versatile resource for developers, enabling them to create innovative and interactive UIs with ease and efficiency. This product will not only improve the productivity of developers but also foster a collaborative community that contributes to the continuous improvement of the platform and its offerings.

Chapter 2

Literature Review

2.1 Existing Systems

2.1.1 Flutter Brick:

Description: Flutter Bricks is a collection of reusable and customizable Flutter components (referred to as "bricks") that streamline development and improve efficiency in building Flutter applications. These bricks provide ready-made solutions for common functionalities and UI elements, such as data handling, animations, navigation, forms, and other aspects of app development.

System Features:

- **Variety of Bricks:** Offers a diverse set of components for different purposes.
- **Easy Integration:** Provides straightforward methods for integrating bricks into projects.
- **Customization:** Bricks can be modified to suit specific project requirements.
- **Performance and Optimization:** Designed for efficient and smooth operation within apps.
- **Community-Driven:** Encourages contributions from the Flutter community for improvement.
- **Comprehensive Documentation:** Offers clear usage guidelines and examples for each brick.
- **Compatibility:** Ensures compatibility with different versions of Flutter and frameworks.
- **Active Maintenance and Updates:** Regularly updates the library with new features and improvements.

In summary, Flutter Bricks simplifies app development by providing developers with a range of pre-built, high-quality components that can be easily integrated, customized, and optimized for performance.

2.1.2 Flutter UI Design:

Description: Flutter UI Design involves creating the user interface (UI) for applications using the Flutter framework, which is known for its fast and flexible development of visually appealing, responsive UIs across mobile, web, and desktop platforms.

System Features:

- **Widget-Based Architecture:** UIs are built using a combination of widgets, the basic building blocks in Flutter.
- **Customizable Widgets:** Provides a wide range of widgets that can be styled and adjusted to create unique UIs.
- **Hot Reload:** Allows developers to instantly see changes in real-time as they modify code, expediting design iterations.
- **Material Design and Cupertino Style:** Supports both Android's Material Design and iOS's Cupertino design principles for cross-platform consistency.
- **Animation and Transition Effects:** Offers powerful tools for creating smooth animations and transitions.
- **State Management:** Various approaches (e.g., Provider, Bloc) enable efficient management of app state for dynamic UIs.
- **Theme Customization:** Developers can apply consistent styles across the app using themes that match branding and design requirements.
- **Cross-Platform Consistency:** Single codebase approach ensures consistent UIs across multiple platforms.

In summary, Flutter UI Design leverages a variety of tools and features within the Flutter framework to create beautiful, efficient, and adaptable UIs for different platforms.

2.1.3 Flutter Gallery:

Description: Flutter Gallery is an official showcase app developed by Google that highlights the capabilities of the Flutter framework. It serves as a reference and learning tool for developers by demonstrating how to use Flutter to create high-quality, visually appealing, and interactive UIs.

System Features:

- **UI Examples:** Provides a wide range of examples showcasing various UI components, widgets, and design patterns.
- **Interactive Demos:** Offers interactive demos that illustrate how to use Flutter's widgets and features effectively.
- **Material and Cupertino Design:** Displays examples of both Material Design (Android) and Cupertino (iOS) design styles for cross-platform development.
- **Source Code Access:** Developers can access the source code for each example, allowing them to learn how the UIs were implemented.
- **Best Practices:** Illustrates best practices in Flutter development, including state management, animations, and responsive design.

- **Consistent Updates:** Regularly updated to include new features and widgets as Flutter evolves.

In summary, Flutter Gallery is a valuable resource for developers looking to learn how to leverage Flutter's capabilities to create stunning and responsive user interfaces.

2.2 Comparative Analysis:

Table: 2.1

Applications Features	Flutter Brick	Flutter UI Design	Flutter Gallery	Our application
Pre-build widget	✓	✓	✓	✓
Responsive design	✓	×	×	✓
User can publish their widget	×	×	×	✓
Basic application layout	✓	×	×	✓
Widget bookmarking	×	×	×	✓

Chapter 3

Requirement Analysis

3.1 External Interface Requirements

3.1.1 User Interfaces:

The user interface will be designed to be intuitive and user-friendly, catering to both novice and experienced developers. It will feature:

- **Widget Browser Interface:** A visually appealing interface allowing users to browse, search, and select widgets easily. This interface will include filters, sorting options, and a preview feature to assist users in finding suitable widgets for their projects.
- **Widget Customization Interface:** An interactive interface enabling users to modify and personalize selected widgets according to their project requirements. This interface will provide options for adjusting widget properties such as size, color, behavior, and more.
- **Code Integration Interface:** A seamless interface facilitating the integration of selected widgets into the user's project codebase. Users will be able to copy widget code snippets with a single click and paste them into their development environment.
- **Documentation Interface:** Accessible documentation for each widget, providing usage guidelines, examples, and best practices. This interface will ensure users have comprehensive information on how to effectively utilize each widget in their projects.

3.1.2 Hardware Interfaces

The software product will primarily interact with standard hardware components commonly found in computing devices such as desktop computers, laptops, and mobile devices. There are no specific hardware dependencies or requirements for the operation of the software.

3.1.3 Software Interfaces

The software product will interface with the following components:

- **Database:** The platform will interact with a database system (e.g. MySQL, MongoDB) to store user accounts, widget metadata, documentation, and user-generated content such as feedback and contributions.
- **Operating System:** The platform will be compatible with various operating systems, including Windows, macOS, and Linux, ensuring accessibility across different devices.

- **Programming Languages and Frameworks:** The platform will support multiple programming languages and frameworks for widget development, including but not limited to Flutter, React, Angular, and Vue.js
- **Version Control System:** Integration with version control systems (e.g. Git) will enable efficient management of website versions, ensuring compatibility and facilitating collaboration among users.

3.1.4 Communications Interfaces

The software product will require the following communication functions:

- **Web Browser:** Users will access the platform via web browsers such as Google Chrome, Mozilla Firefox, and Safari. The platform will be optimized for compatibility with modern web standards and browsers.
- **Network Server Communications Protocols:** Communication between the client (user's device) and the server hosting the platform will utilize standard protocols such as HTTP.
- **Message Formatting:** Data exchanged between the client and server will adhere to standard formats such as JSON (JavaScript Object Notation) for efficient data transmission and parsing.
- **Communication Security:** The platform will implement secure communication protocols (e.g., HTTPS) to protect user data and ensure confidentiality, integrity, and authenticity during transmission.
- **Data Transfer Rates:** The platform will aim for optimal data transfer rates to minimize latency and ensure responsive user experiences, especially during widget browsing and integration processes.
- **Synchronization Mechanisms:** Synchronization mechanisms will be employed to ensure consistency between client and server data, particularly for user preferences, widget selections, and user-generated content.

3.2 Functional Requirements

Table: 3.1

ID	Requirement
REQ-1	User authentication
REQ-2	User data management
REQ-3	Widget category management
REQ-4	Templates
REQ-5	Publish widget

3.3 Other Nonfunctional Requirements

3.3.1 Performance Requirements

- **Response Time:** The platform should respond to user interactions (e.g., widget browsing, selection, customization) within a maximum of 2 seconds to ensure a seamless and responsive user experience.
- **Scalability:** The system should be able to handle a growing user base and increasing widget repository size without significant degradation in performance. It should scale horizontally and vertically to accommodate increased load and demand.

3.3.2 Safety Requirements

- **Data Integrity:** Ensure the integrity of user data, widget metadata, and other sensitive information stored within the system. Implement measures such as data encryption, access controls, and regular backups to prevent data loss or corruption.
- **User Authentication:** Implement robust user authentication mechanisms to prevent unauthorized access to user accounts and sensitive data. Utilize secure authentication protocols (e.g. OAuth 2.0 or JWT) and enforce password complexity requirements.

3.3.3 Security Requirements

- **Secure Data Transmission:** All communication between the client and server should be encrypted using industry-standard encryption protocols (e.g. TLS/SSL) to protect against eavesdropping and data interception.
- **Protection Against Common Attacks:** Implement security measures such as input validation, output encoding, and parameterized queries to mitigate risks associated with common web application vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
- **Access Control:** Enforce role-based access control (RBAC) to restrict access to sensitive functionalities and data based on user roles and permissions. Only authorized users should be able to perform actions such as publishing widgets or managing user data.

3.3.4 Software Quality Attributes

- **Usability:** Prioritize usability by designing intuitive user interfaces, providing clear documentation, and offering guided tutorials to facilitate user onboarding and adoption of the platform.
- **Maintainability:** Structure the codebase using modular and well-documented components, adhering to coding standards and best practices. This will enable easier maintenance, updates, and future enhancements to the platform.
- **Reliability:** Ensure the reliability of the platform by conducting thorough testing (unit testing, integration testing, etc.) to identify and address any bugs or issues. Implement error handling mechanisms to gracefully handle unexpected errors and prevent system crashes or data loss.
- **Flexibility:** Design the system to be flexible and adaptable to changing requirements and technologies. Use design patterns and architectural principles that allow for easy extension and customization, enabling the platform to evolve over time.
- **Interoperability:** Ensure compatibility with a wide range of devices, browsers, and operating systems to maximize the platform's reach and usability across different environments. Adhere to industry standards and protocols for seamless integration with external systems and services.
- **Portability:** Design the platform to be portable across different hosting environments (e.g., cloud, on-premises) and deployment configurations. Minimize dependencies on specific hardware or software components to enable easy deployment and migration.

3.4 Cost Estimation

Table: 3.2

S.No	Project Expenditure	Cost in Rupees
1	Domain Name and Hosting	
	Domain Name	4000
	Hosting	10000
	Sub Total	14,000
2	Printing	
	FYP Wall Poster	2600
	FYP Standee	1840
	FYP Brochure	2000
	FYP Report 3 Copies	6000
	2 USBs	2000
	Sub Total	14,440
	Grand Total	28,440

System Design

4.1 System Architecture Diagram

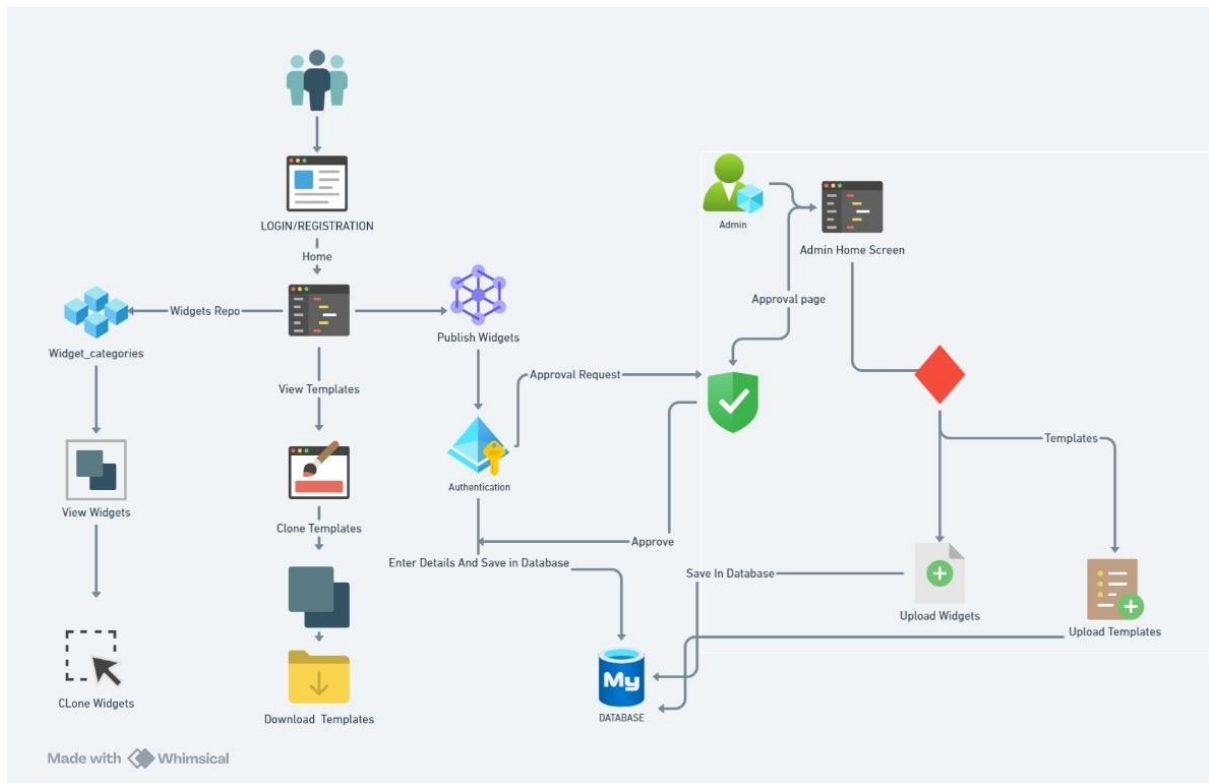


Figure: 4.1

4.2 System Operations

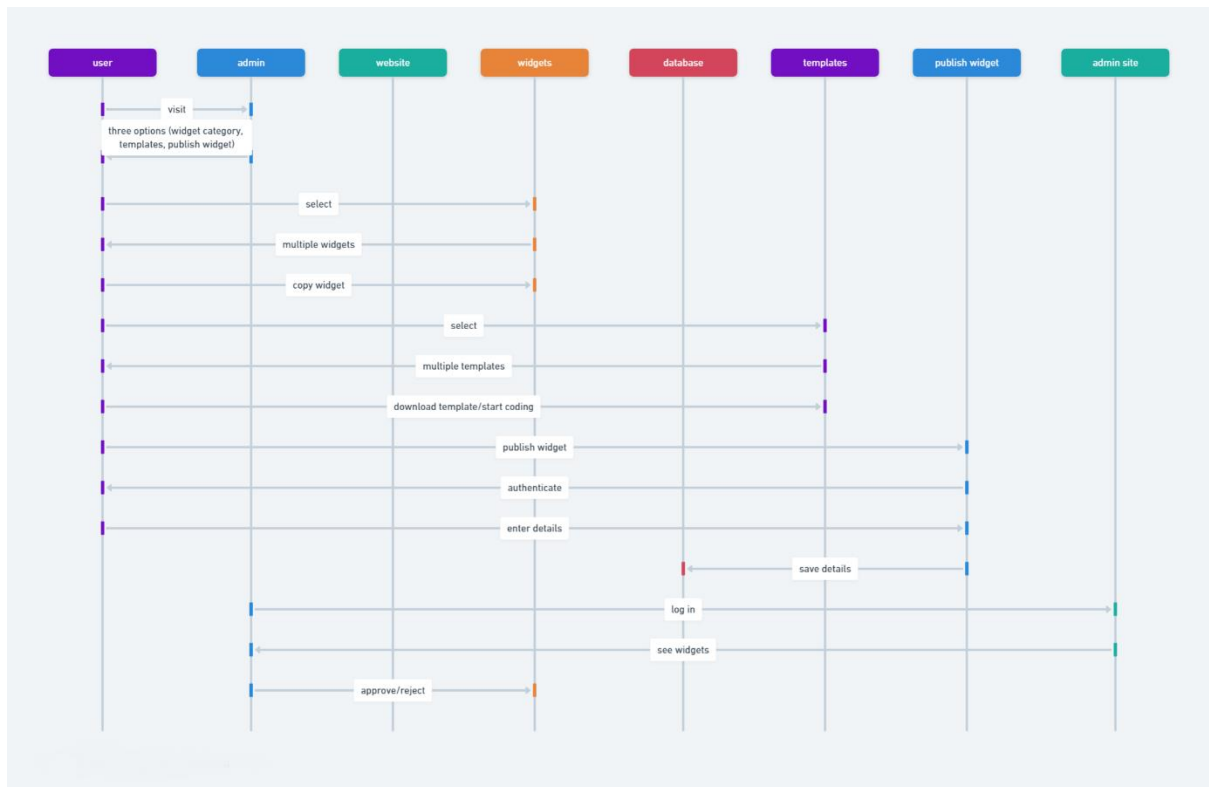


Figure: 4.2

4.3 System Model

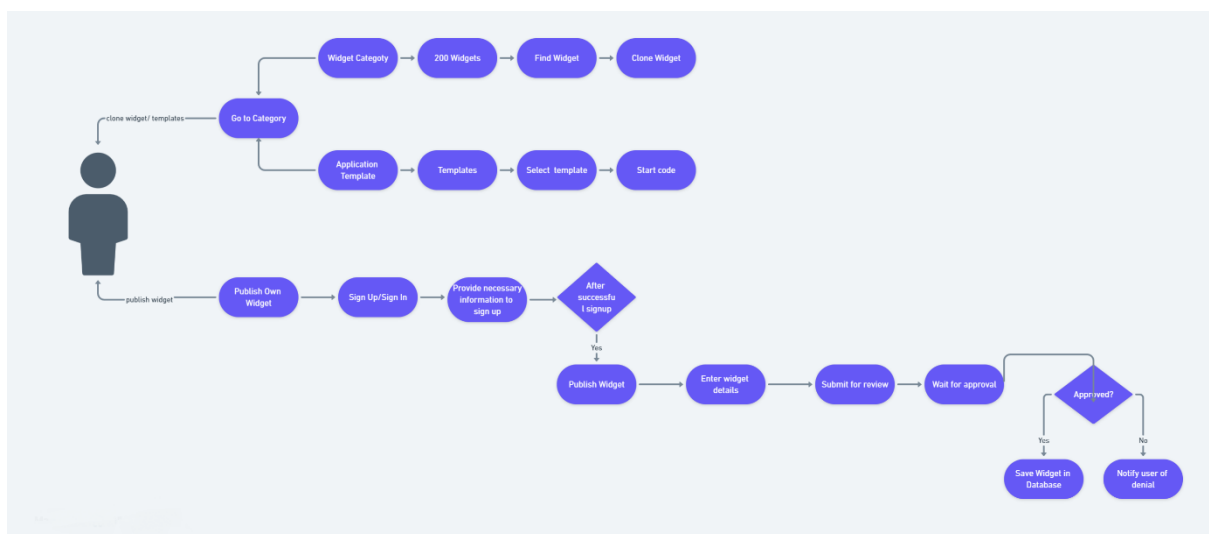


Figure: 4.3

4.4 Object Model

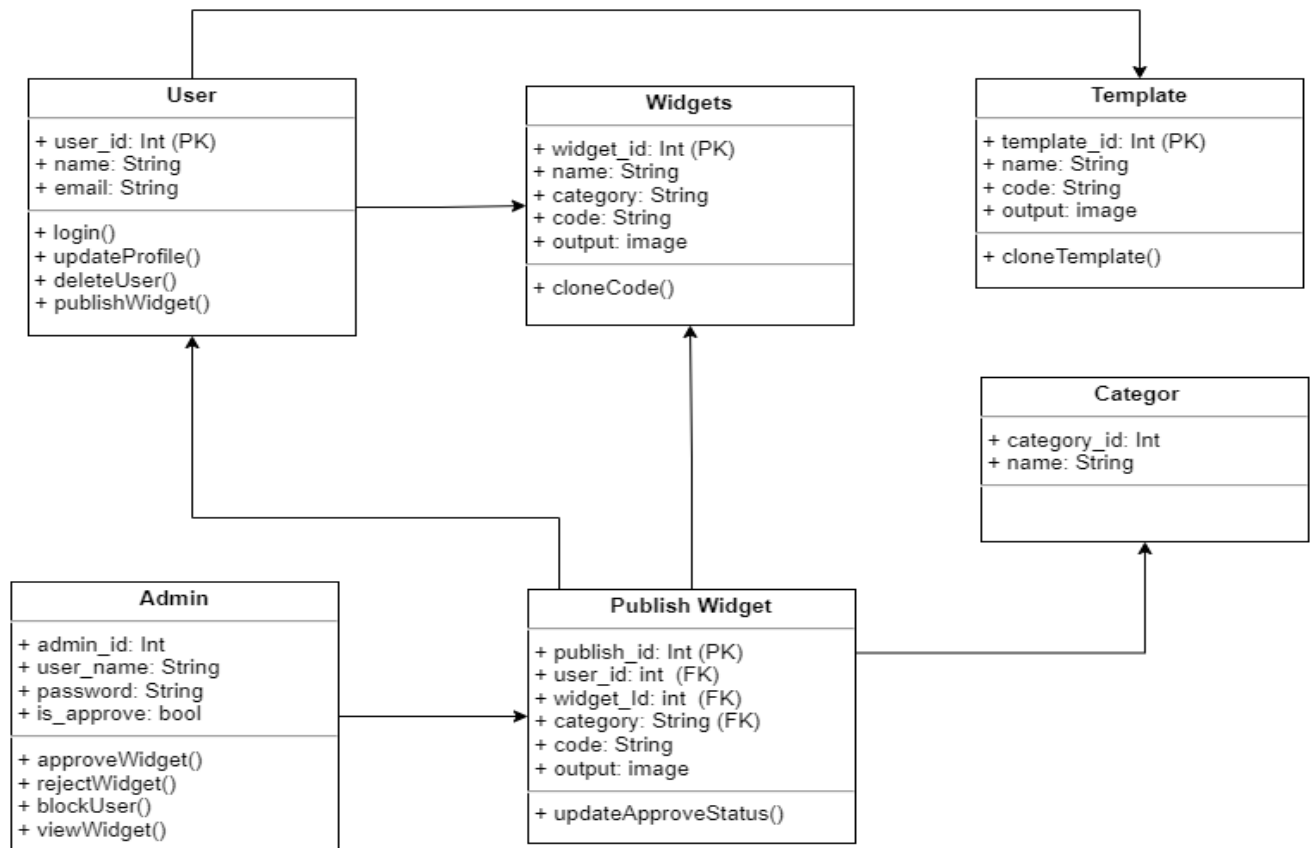


Figure: 4.4

4.5 Data Model

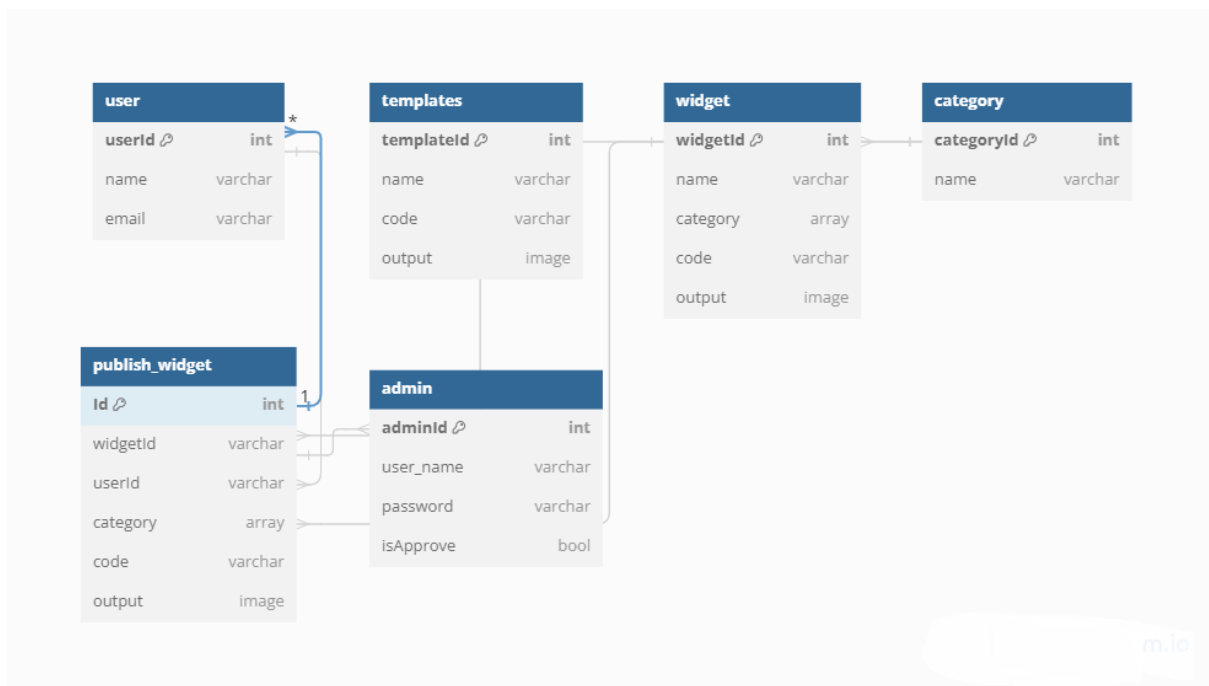


Figure: 4.5

4.6 User Interface Design

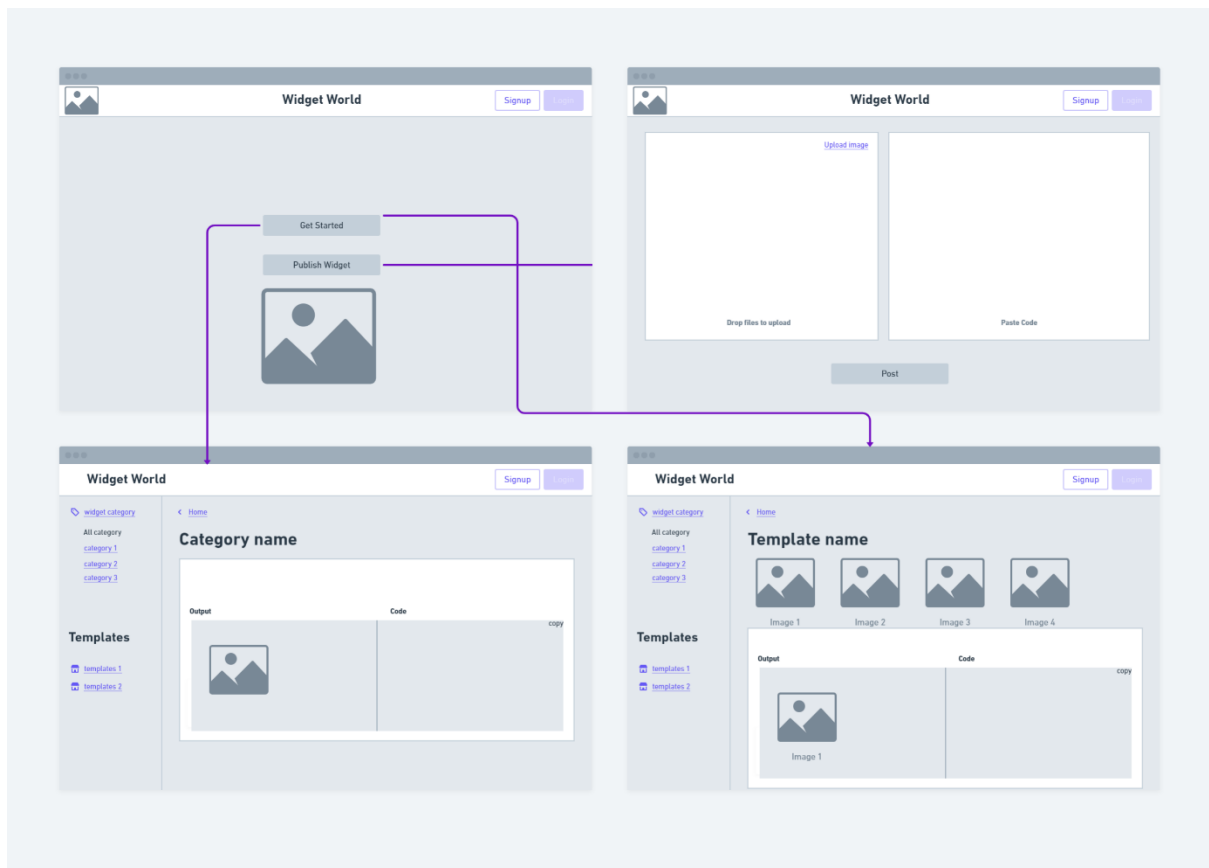


Figure: 4.6

Chapter 5

Business Model

5.1 Business Model Canvas

The Business Model Canvas for the Widget World project focuses on developing a web app with Flutter widgets to streamline mobile app development. Below is a detailed canvas addressing each section:

5.1.1 Key Partners

- **Flutter Community and Developers:** Essential for contributing to the widget library and providing feedback.
- **Open-Source Contributors:** Help enhance the widget library and contribute new widgets.
- **Cloud Service Providers:** Providers such as AWS, Google Cloud, for hosting the web app.
- **Educational Platforms:** Platforms like Udemy, Coursera, or YouTube for distributing tutorials and educational content.
- **UI/UX Design Tool Providers:** Tools like Figma, Adobe XD for designing and prototyping widgets.

5.1.2 Key Activities

- **Development and Maintenance:** Creating and updating the Flutter widget library.
- **Web App Development:** Building and enhancing the web application.
- **Community Management:** Engaging with and managing the community of developers and contributors.
- **Content Creation:** Producing documentation, tutorials, and educational materials.
- **Customization Tools:** Developing tools that allow users to modify and personalize widgets.
- **User Support:** Providing assistance and resolving issues for users.
- **Marketing and Outreach:** Promoting the platform and engaging with potential users through various channels.
- **Quality Assurance:** Testing widgets and the web app to ensure functionality and performance.

5.1.3 Key Resources

- **Domain Name:** The web address for your platform, which is crucial for online presence and accessibility.
- **Hosting Infrastructure:** Servers and cloud services for hosting the web app and managing traffic.
- **Development Tools:** Software and tools required for coding, designing, and testing the widget library and web app.
- **Design Assets:** UI/UX designs, branding materials, and prototypes created using tools like Figma or Adobe XD.
- **Data Storage:** Solutions for storing user data, widget information, and other critical data.
- **Technical Documentation:** Guides and manuals for both internal use and customer support.
- **Customer Support Systems:** Platforms for managing user queries, feedback, and support requests.
- **Marketing Channels:** Accounts and tools for social media, email marketing, and other promotional activities.

5.1.4 Value Propositions

- **Free of Cost:** The widget library is available at no charge, reducing financial barriers for developers.
- **Streamlined Development:** Offers pre-built, customizable widgets that significantly speed up the mobile app development process.
- **Ease of Use:** Provides an intuitive platform for browsing, selecting, and integrating widgets, making development simpler and more efficient.
- **Comprehensive Documentation:** Includes detailed guides and tutorials to help developers easily implement and use the widgets.
- **Community Collaboration:** Fosters a community-driven approach where users can contribute, share feedback, and improve the widget library collectively.
- **Enhanced User Experience:** Delivers high-quality, visually appealing widgets that enhance the overall user experience of mobile applications.

5.1.5 Customer Relationships

- **Customer Support:**
 - **Dedicated Support Channels:** Offer various support options such as email, live chat, or a helpdesk to assist users with any issues or questions they have.
 - **Knowledge Base:** Provide a comprehensive knowledge base or FAQ section on the website to help users find answers to common questions and troubleshoot problems on their own.

- **Feedback Forms:**
 - **User Feedback Collection:** Implement feedback forms or surveys to gather user opinions, suggestions, and experiences with the widget library and web app.
 - **Continuous Improvement:** Use the feedback collected to make informed updates and improvements to the widget library and platform, ensuring it evolves according to user needs and preferences.

5.1.6 Channels

- **Website:** The primary channel where users can access the Flutter widget library. The website serves as the main platform for browsing, selecting, and integrating widgets into mobile app projects. It also provides access to documentation, support, and updates related to the widget library.

5.1.7 Cost Structure

- **Domain and Hosting Fees:**
 - **Domain Registration:** Annual fee for maintaining the domain name.
 - **Web Hosting:** Costs for cloud hosting services and infrastructure to support the website.

5.1.8 Revenue Streams

- **Ad Revenue:**
 - **Display Ads:** Income from placing advertisements on the website or within the web app. Revenue is generated through impressions or clicks on these ads.
- **Subscription Model:**
 - **Premium Features:** Offering advanced or additional features of the widget library for a subscription fee. This could include exclusive widgets, enhanced customization options, or priority support.

5.1.9 Customer Segments:

- **Independent Developers:**
 - **Description:** Individual developers looking for efficient tools to speed up their app development process.
 - **Needs:** Easy-to-use widgets, comprehensive documentation, and customizable options.

- **Development Teams:**
 - **Description:** Teams working on mobile app projects within companies or startups.
 - **Needs:** Reliable and scalable widgets, collaboration tools, and integration support.
- **Startups:**
 - **Description:** New businesses that need cost-effective solutions for building mobile applications quickly.
 - **Needs:** Free or low-cost widgets, rapid development tools, and scalability options.
- **Educational Institutions:**
 - **Description:** Schools, universities, and coding bootcamps teaching mobile development.
 - **Needs:** Educational resources, tutorials, and a range of widgets for teaching and learning purposes.
- **Freelancers:**
 - **Description:** Freelance developers and consultants working on various client projects.
 - **Needs:** Versatile widgets, ease of integration, and efficient development tools.
- **Tech Enthusiasts:**
 - **Description:** Hobbyists and technology enthusiasts interested in experimenting with new tools and technologies.
 - **Needs:** Innovative and customizable widgets, and access to a broad range of features.

Chapter 6

System Development Methodology

6.1 Development Methodology

Waterfall Methodology:

Description: The Waterfall methodology is a linear and sequential approach where each phase of development is completed before moving on to the next. The phases are: Requirements → Design → Implementation → Verification → Maintenance.

Justification for Use in Widget World:

- **Clear Requirements:** The Widget World project has well-defined goals and requirements, such as developing a web app for Flutter widgets with specific functionalities and user features. Since these requirements are expected to remain stable throughout the project, Waterfall's structured approach is suitable.
- **Structured Approach:** Waterfall provides a clear, step-by-step process. For Widget World, this means each phase—requirements gathering, design, development, testing, and maintenance—can be executed systematically, ensuring that each aspect of the widget library and web app is thoroughly addressed in sequence.
- **Documentation:** The Waterfall methodology emphasizes thorough documentation at every stage. For Widget World, this ensures that all aspects of the widget library and web app are well-documented, from initial requirements to final implementation, facilitating a clear understanding of the project scope and making it easier to track progress and manage changes.

Using Waterfall for Widget World aligns well with the project's needs for a clear, linear development process with stable requirements and a focus on comprehensive documentation.

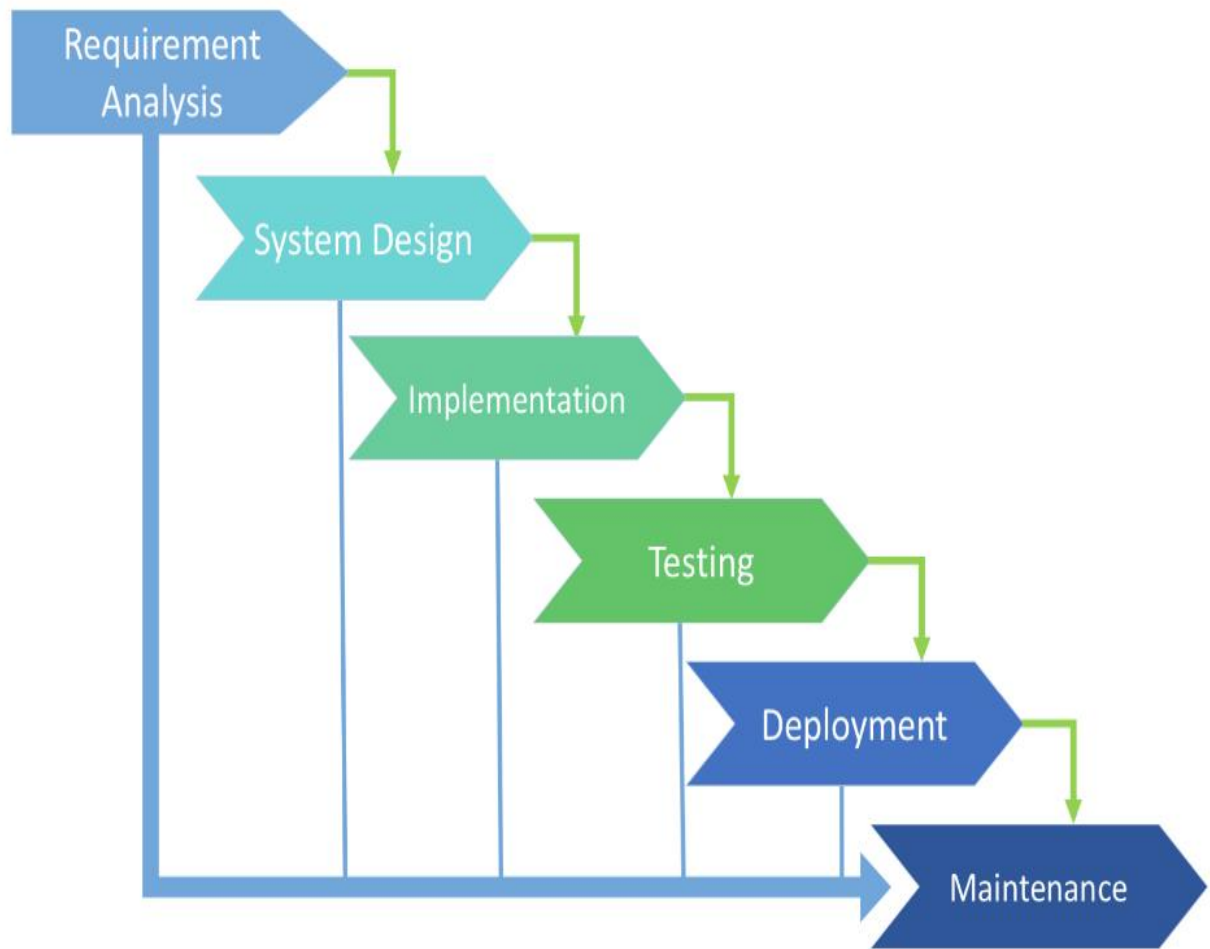


Figure: 6.1

Waterfall Model

6.2 Key Milestone

Table: 6.1

S.no	Milestone	Completion Month
1	System Planning	Month 1
2	Requirement gathering and analysis	Month 2
3	Widget Library Design	Month 3
4	Widget Library Implementation (Part 1)	Month 4
5	Widget Library Implementation (Part 2)	Month 5
6	website design (Part 1)	Month 6
7	website design (Part 2)	Month 7
8	API Development	Month 8
9	API Deployment	Month 9
10	Web Domain and Hosting	Month 10
11	Project Completion and Maintenance	Month 11
12	FYP Report Writing	Month 12

6.3 Project Plan with Gantt Chart



Figure: 6.2

6.4 Tools Used for Development

To develop this project, there are several tools and requirements needs to be used to run the system. Below is the minimum requirement and tools required:

Personal computers with Windows platform, 4 GB RAM (minimum), 500 GB hard-disk space, including 350 MB of available space on the hard disk that contains the operating system – to store all files and documents regarding the system

6.5 System Implementation

1. Frontend Development:

- **User Interface (UI):**
 - Develop a responsive and visually appealing user interface using Flutter, optimized for both web and mobile views.
 - Implement a user-friendly platform with an intuitive design, allowing users to browse, select, and modify widgets with ease.
- **Widget Integration:**
 - Create a catalog of widgets displayed in a grid or list format, allowing users to preview and select desired widgets.
 - Enable real-time customization of widgets through a built-in editor, allowing users to adjust properties such as size, color, and functionality.
 - Implement a “copy code” feature, allowing users to copy the code of the selected widget directly into their project codebase.
- **Search and Navigation:**
 - Implement a robust search function allowing users to find specific widgets quickly.
 - Include categorization and filtering options to enhance navigation through the widget library.

2. Backend Development

- **Database Management:**
 - Set up a database to store widget data, user profiles, contributions, and version control information.
 - Implement a version control system for widgets, ensuring compatibility with different Flutter versions and frameworks.
- **APIs:**
 - Develop RESTful APIs to handle CRUD (Create, Read, Update, Delete) operations for widgets and user data.
 - Integrate APIs for real-time data synchronization between the frontend and backend, ensuring seamless updates and interactions.
- **User Authentication and Profiles:**
 - Implement user authentication mechanisms, allowing users to create accounts, save preferences, and contribute to the widget library.
 - Create user profiles that store their selected widgets, contributions, and feedback.

3. Community and Collaboration Features

- **Contribution System:**
 - Develop a system that allows users to upload and share their custom widgets, including code and documentation.
 - Implement moderation tools to ensure quality and compatibility of user-contributed widgets.
- **Feedback and Improvement:**
 - Enable users to provide feedback on widgets, including ratings, comments, and suggestions for improvements.
 - Implement a system to track and integrate user feedback into future versions of widgets.

4. Documentation and Tutorials

- **Comprehensive Documentation:**
 - Provide detailed documentation for each widget, including code examples, customization options, and integration guidelines.
- **Tutorials and Guides:**
 - Develop tutorials to guide users through common use cases, such as integrating widgets into their projects and customizing them to fit specific needs.
 - Include video tutorials and interactive examples to enhance learning and engagement.

5. Version Control and Compatibility

- **Version Management:**
 - Implement a version management system to track updates and changes to widgets, ensuring backward compatibility.
- **Compatibility Checks:**
 - Develop automated tests to verify widget compatibility with different versions of Flutter and other relevant frameworks.
 - Provide warnings or notifications if a selected widget may have compatibility issues with the user's current project setup.

6. Deployment and Maintenance

- **Web Hosting:**
 - Deploy the web app on a reliable hosting platform, ensuring high availability and performance.
- **Ongoing Maintenance:**
 - Set up continuous integration and continuous deployment (CI/CD) pipelines to automate testing and deployment of updates.
 - Monitor the platform for bugs, security vulnerabilities, and performance issues, implementing fixes and improvements as needed.

7. Template Library

- **Pre-Built Templates:**
 - Develop and include a variety of pre-built templates for complete mobile applications, covering different use cases and industries.
 - Allow users to customize these templates and integrate widgets from the library to create unique applications.
- **Template Management:**
 - Implement version control and updates for templates, ensuring they stay relevant and compatible with the latest Flutter features and best practices.

Chapter 7

Testing and Evaluation

7.1 Introduction

7.1.1 Purpose

This document outlines the system testing plan for the Final Year Project (FYP) focused on accelerating mobile application development using Flutter widgets. The testing strategy described here aims to ensure that the developed web app meets all specified business requirements and is ready for release.

7.1.2 Objectives

- **Compliance with Requirements:** Verify that the web app meets all project objectives, specifications, and business rules.
- **Support for Business Functions:** Ensure that the web app supports the intended business functions and adheres to required software standards.
- **User Acceptance Testing Readiness:** Confirm that the application meets the criteria for User Acceptance Testing (UAT).

7.2 Test Methodology

The Black Box Testing method will be employed to test the system. This approach will focus on validating the functionality of the web app by evaluating its inputs and outputs without examining the internal code structure.

7.3 Test Plan

Table: 7.1

ID	Requirement
REQ-1	User authentication
REQ-2	User data management
REQ-3	Widget category management
REQ-4	Templates
REQ-5	Publish widget

7.4 Test Approach

The testing approach will include the following phases:

- **Functional Testing:** Validate that all Flutter widgets within the web app function as expected, including browsing, selection, integration, modification, and personalization features.
- **Usability Testing:** Assess the user interface and experience, ensuring that the platform is intuitive and easy to navigate.
- **Performance Testing:** Evaluate the web app's performance under different conditions, including load and stress testing, to ensure it handles high usage effectively.
- **Compatibility Testing:** Verify that the web app and its widgets are compatible with various devices, browsers, and operating systems.
- **Security Testing:** Identify and mitigate any security vulnerabilities to ensure the web app is secure.

7.5 Test Environment

The test environment will include:

- **Hardware:**
 - Test Servers with appropriate specifications to simulate real-world usage.
 - Test machines with different configurations to assess performance across various devices.
- **Software:**
 - Operating Systems: Windows, macOS, Linux
 - Web Browsers: Chrome, Firefox, Safari, Edge
 - Flutter Framework: Latest stable version
 - Database: MySQL, Firebase
 - Testing Tools: Selenium for automated testing, JIRA for issue tracking

7.6 Test Entrance Criteria

Before testing can commence, the following prerequisites must be met:

- Completion of unit testing by the development team
- All core functionalities must be implemented
- Test environment setup and configuration
- Approval of the testing plan by the project stakeholders

7.7 Testing Acceptance Criteria

- All major functionalities of the web app should work as intended.
- The pass percentage for test cases should be above 95%.
- There should be no critical or high-severity bugs remaining in the system.

7.8 Test Cases

Table: 7.2

Test Case ID	Test Objective	Test Data	Expected Result	Actual Result	Test Status Pass / Fail	Bug ID
TC1	Ensure that the user can successfully log in with valid credentials.	Username: testuser Password: ValidPassword123	User should be successfully authenticated and redirected to the dashboard.	Login Successful	Pass	N/A
TC 2	Ensure the system handles incorrect login attempts properly.	Username: invalid user Password: InvalidPassword456	User should receive an error message indicating invalid login credentials.	Login successful	Fail	BG-1
TC 3	Ensure that new user data can be created and stored in the database	User Name: NewUser Email: newuser@example.com Role: Developer	New user should be created and data stored in the database.	User Created	Pass	N/A
TC 4	Ensure existing user data can be updated.	Existing User: ExistingUser Updated Email: updateduser@example.com	User details should be updated and reflected in the database.	User information stored in database	Pass	N/A
TC 5	Verify that widget categories can be managed effectively	New Category Name: UI Components Edit Category Name: UI Elements Delete Category: OldCategory	Categories should be added, updated, and deleted as per the user's actions.	Categories not added, updated, and deleted as per the user's actions.	Fail	BG-2
TC6	Ensure that templates can be created and applied to projects.	Template Name: App Template Template Details: Basic app layout with navigation drawer Project: SampleProject	Template should be created and correctly applied to the project	Template created successfully	Pass	N/A

	Text font on reports is incorrect (e.g., font style used in documentation or on the platform is not as intended).
--	---

7.9.3 Bug Priority Definitions

Table: 7.4

Levels	Definitions
High	Must be fixed in any of the upcoming builds and should be included in the release (e.g., critical bugs affecting user authentication or widget publishing).
Medium	May be fixed after the release or in the next release (e.g., medium-level bugs like incorrect form navigation).
Low	May or may not be fixed at all, depending on impact (e.g., minor cosmetic issues)

7.9.4 Bug Life Cycle

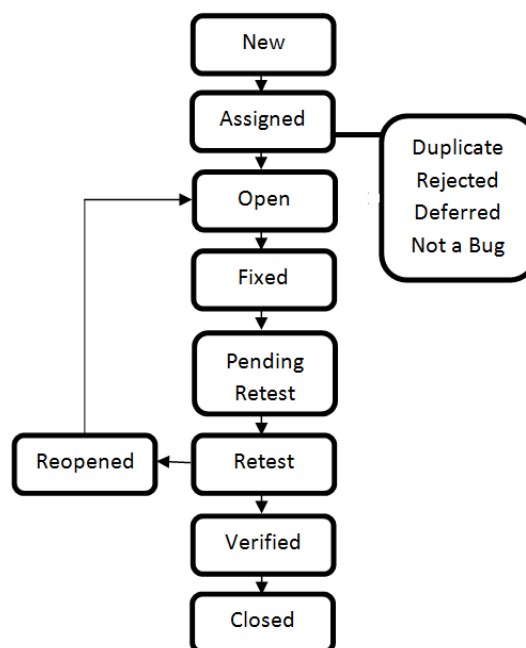


Figure: 7.1

Assigned: After the tester has posted the bug, the lead of the tester approves that the bug is genuine and he assigns the bug to corresponding developer and the developer team. Its state given as assigned.

Open: At this state the developer has started analyzing and working on the defect fix.

Fixed: When developer makes necessary code changes and verifies the changes then he/she can make bug status as 'Fixed' and the bug is passed to testing team.

Pending retest: After fixing the defect the developer has given that particular code for retesting to the tester. Here the testing is pending on the testers end. Hence its status is pending retest.

Retest: At this stage the tester do the retesting of the changed code which developer has given to him to check whether the defect got fixed or not.

Verified: The tester tests the bug again after it got fixed by the developer. If the bug is not present in the software, he approves that the bug is fixed and changes the status to "verified".

Reopen: If the bug still exists even after the bug is fixed by the developer, the tester changes the status to "reopened". The bug goes through the life cycle once again.

Closed: Once the bug is fixed, it is tested by the tester. If the tester feels that the bug no longer exists in the software, he changes the status of the bug to "closed". This state means that the bug is fixed, tested and approved.

Duplicate: If the bug is repeated twice or the two bugs mention the same concept of the bug, then one bug status is changed to "duplicate".

Rejected: If the developer feels that the bug is not genuine, he rejects the bug. Then the state of the bug is changed to "rejected".

Deferred: The bug, changed to deferred state means the bug is expected to be fixed in next releases. The reasons for changing the bug to this state have many factors. Some of them are priority of the bug may be low, lack of time for the release or the bug may not have major effect on the software.

Not a bug: The state given as "Not a bug" if there is no change in the functionality of the application. For an example: If customer asks for some change in the look and field of the application like change of color of some text then it is not a bug but just some change in the looks of the application

7.9.5 Bug Reports

Table: 7.5

Bug ID	Test Case ID	Bug Description	Reported By	Status	Fixed by	Severity	Priority
BG-1	TC2	Login successful after entering with wrong credential	Talha	Fixed	Hammad	Critical	High
BG-2	TC5	widget categories had not managed effectively	Khubaib	Fixed	Obaid	Medium	Medium
BG-3	TC7	widgets published but not available to other users.	Khubaib	Fixed	Hammad	Low	Low

7.10 User Acceptance Testing

Table: 7.6

User# 1 Review:			
<i>"This platform has completely transformed the way I develop mobile apps. The variety of Flutter widgets available is impressive, and the integration process is seamless. I've saved both time and money by using this app instead of subscribing to costly services. It's fantastic to see such a robust, free, and open-source solution!"</i>			
Name	Muhammad Affan	Signature	

Table No 7.7

User# 2 Review:			
<i>"As a freelance developer, cost-effectiveness is crucial for me. This web app offers an incredible selection of widgets that are easy to customize and integrate. The user interface is intuitive, and the design is sleek. I've been able to quickly adapt the widgets to my projects, making development much more efficient."</i>			
Name	Abdul Samad	Signature	

Table No 7.8

User# 3 Review:			
<i>"The platform's focus on being open-source and community-driven is truly commendable. I love how the community can contribute and improve the widget library, making it a dynamic and evolving resource. The comprehensive documentation is a huge plus, making it easy even for beginners to dive in and start developing. This app has significantly enhanced my workflow and creativity."</i>			
Name	Faizan Ahmed	Signature	

Chapter 8

Result and Discussion

Result:

The development of key components in our Flutter-based web app, including the login screen, registration screen, widget publication interface, template section, and user dashboard, has been completed successfully. Each feature has been rigorously tested for functionality, responsiveness, and user experience across multiple devices and screen sizes. The results indicate that the web app performs well in all areas, providing users with a seamless, intuitive, and efficient tool for mobile application development.

- **Login Screen:** The login screen functions smoothly, with secure user authentication, robust error handling, and a responsive design that adjusts seamlessly to various screen sizes. Users can log in quickly and securely, and the system effectively manages incorrect login attempts with clear, helpful error messages.
- **Registration Screen:** The registration process is straightforward and secure, allowing users to sign up easily across all devices. The form is simple and user-friendly, with validation mechanisms in place to ensure the integrity of the data entered. The screen is also responsive, maintaining its usability and aesthetic appeal on smartphones, tablets, and desktops.
- **Widget Publication Screen:** The widget publication interface offers a streamlined and secure process for submitting new widgets. Users can easily manage their submissions, with options to edit or delete widgets as needed. The interface is designed to be intuitive, making it easy for users to navigate and perform necessary actions without confusion.
- **Template Section:** The template section provides a variety of pre-built, responsive templates that users can quickly copy and implement into their projects. This feature is particularly useful for developers looking to save time, as it eliminates the need for customization while still offering a range of design options. The templates are responsive, ensuring they work well across all devices.
- **User Dashboard:** The user dashboard offers a comprehensive view of the user's widgets, allowing them to update, delete, or manage their widgets with ease. The dashboard is designed to be user-friendly, providing clear options and a simple layout that facilitates efficient project management.

Discussion:

The successful implementation and testing of these features mark significant milestones in the development of our Flutter-based web app. Each component has been designed with the user experience in mind, ensuring that the app is not only functional but also intuitive and easy to use.

- **Login and Registration Screens:** The login and registration screens are critical entry points for users, and their design has been carefully considered to balance security with usability. The use of standard security practices, such as encryption and secure communication protocols, ensures that user data is protected. At the same time, the screens are designed to be straightforward, minimizing the time and effort required for users to access the app. The responsive design further enhances the user experience by ensuring that the screens are accessible and functional on a variety of devices.
- **Widget Publication and User Dashboard:** The widget publication screen and user dashboard are central to the app's functionality, as they enable users to manage their widgets effectively. The publication interface is designed to be as simple as possible, allowing users to submit and manage their widgets without encountering unnecessary complexity. The user dashboard complements this by offering a clear overview of the user's widgets, with easy-to-access options for updating or deleting them. These features collectively empower users to take full control of their projects, ensuring that they can manage their resources efficiently and effectively.
- **Template Section:** The template section is a valuable addition to the app, offering users a quick and easy way to implement designs without the need for customization. This feature is particularly useful for developers who need to produce results quickly, as it provides ready-made solutions that can be copied and pasted into projects. The templates are designed to be responsive, ensuring that they work well on a variety of devices and screen sizes. This flexibility is essential in today's mobile-first world, where users expect applications to function seamlessly across different platforms.
- **Overall Usability and Responsiveness:** One of the key strengths of the app is its focus on usability and responsiveness. Every screen and interface has been designed to be user-friendly, with clear instructions, intuitive layouts, and immediate feedback. This attention to detail ensures that users can navigate the app with ease, regardless of their level of technical expertise. The responsiveness of the app further enhances its usability, as it ensures that the app remains functional and visually appealing across all devices, from smartphones to desktops.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This Final Year Project (FYP) has successfully addressed the need for accelerating mobile application development by leveraging the power of Flutter widgets. By creating a comprehensive, user-friendly web app that integrates innovative features, we have streamlined the development process, making it faster, more efficient, and cost-effective. The project has delivered a versatile platform that not only meets but exceeds current market standards, providing developers with a robust toolset that enhances their productivity and creativity.

Our commitment to open-source development has ensured that the platform is accessible to a wide range of developers, eliminating the need for expensive subscriptions and offering a free alternative that democratizes mobile app development. The introduction of a new Flutter widget library, alongside the capability for users to modify and personalize widgets, fosters a highly adaptable environment that caters to diverse project requirements.

Moreover, by incorporating community-driven contributions and maintaining comprehensive documentation, the project has laid the foundation for a continuously evolving ecosystem that supports collaboration and innovation. The implementation of version management and compatibility with different programming languages and frameworks further strengthens the platform's utility and relevance in the dynamic landscape of mobile app development.

In conclusion, this project not only meets its objectives but also sets a new benchmark for mobile application development tools, empowering developers with the resources they need to create engaging, interactive, and competitive Flutter applications. Through this FYP, we have made a significant contribution to the mobile development community, aligning with the ever-evolving demands of the digital era.

9.2 Future Work

As we look toward the future of our platform, one of the key areas for development is integrating machine learning to enhance the quality and integrity of the widgets being published. Here are the proposed advancements:

1. Machine Learning for Widget Validation:

We will implement machine learning algorithms to automatically verify that the published items on our platform are indeed Flutter widgets. This system will analyze various characteristics and patterns of the submitted code to ensure it adheres to the expected widget structures and functionalities. By doing so, we can prevent non-widget content from being uploaded, maintaining the platform's integrity and usability.

2. Automated Classification System:

The machine learning model will classify and categorize widgets based on their functionalities and attributes. This automated classification will improve the organization of widgets, making it easier for users to find relevant widgets quickly and efficiently.

3. Anomaly Detection:

We will incorporate anomaly detection mechanisms to identify unusual patterns or deviations in the widget submissions. This will help in detecting potentially harmful or incorrect code that could affect the platform's performance or security.

4. Enhanced User Feedback:

Machine learning can be used to analyze user interactions and feedback on widgets to continually refine and improve the widget validation process. This feedback loop will help in identifying areas for improvement and ensuring that the widgets meet the users' needs and expectations.

5. Adaptive Learning:

The system will employ adaptive learning techniques to continuously improve its accuracy and effectiveness in widget validation. As more widgets are processed and more data is collected, the machine learning models will be updated to better recognize and classify widgets.

By integrating machine learning into the widget validation process, we aim to create a more robust and reliable platform, ensuring that only genuine Flutter widgets are available to developers. This future work will significantly enhance the platform's functionality and user experience, aligning with our commitment to innovation and excellence in mobile application development.

References:

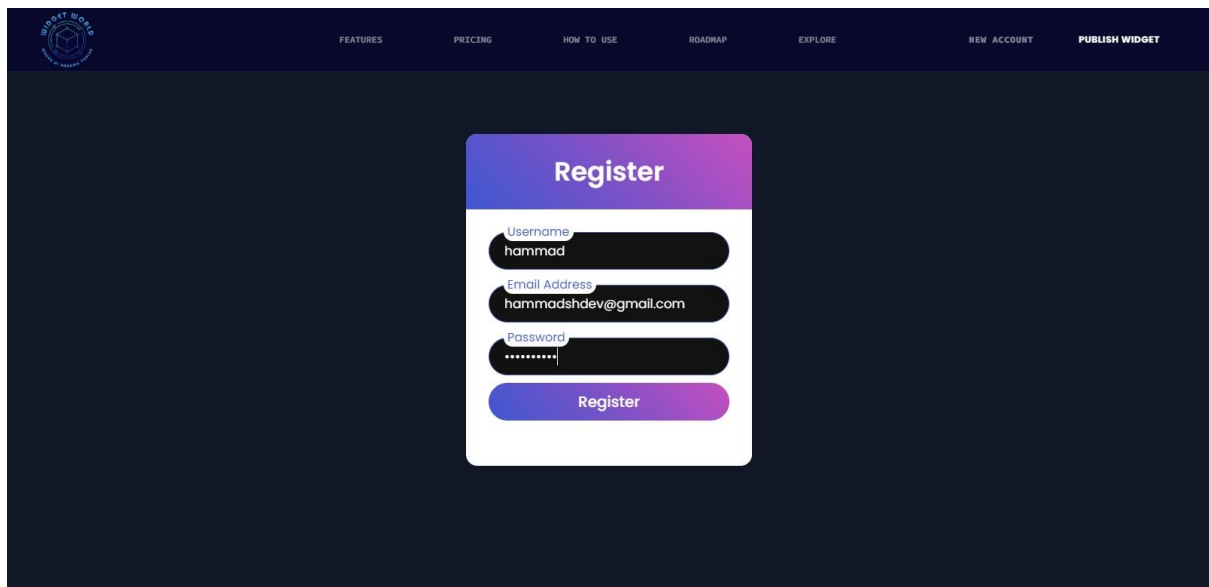
- a) **Ray Wenderlich Tutorials** (2020). "Website has various tutorials on Flutter that cover different aspects of mobile app development." Flutter UI design
<https://flutterui.design/>
- b) **Richter Et Al** (2018). "This research paper delves into the use of Flutter for scalable location-based services in cloud computing. It might provide insights into Flutter's capabilities beyond mobile app development. Flutter Bricks
<https://www.flutterbricks.com/>

Appendices

Appendix 1: User Manual

Registration Screen

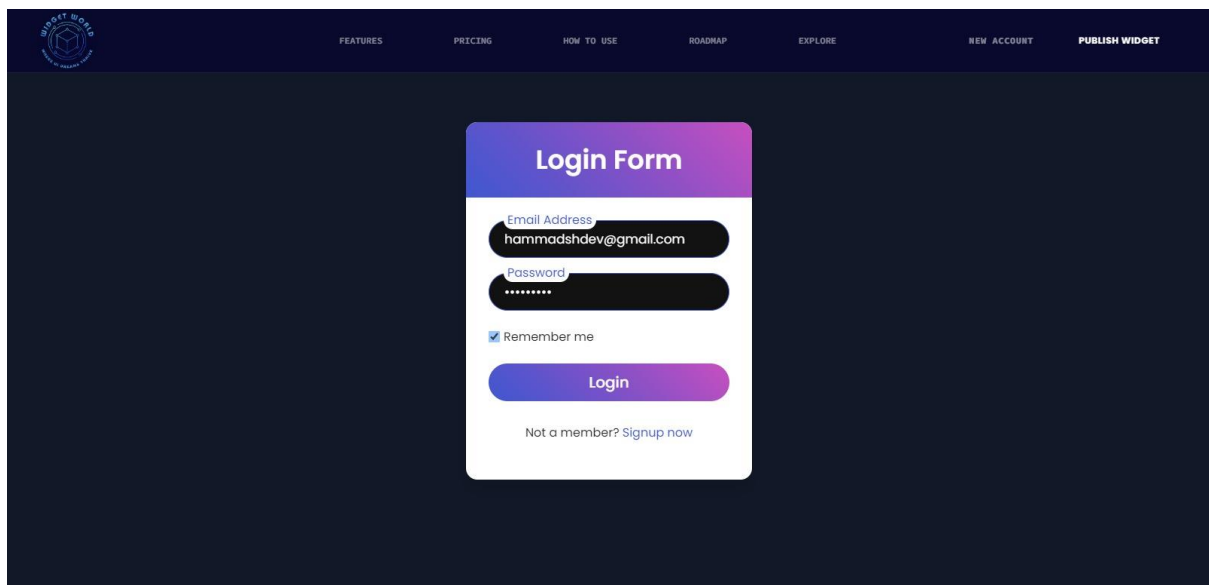
The registration screen is designed to be straightforward, secure, and responsive, ensuring a smooth and efficient user sign-up process across all devices.



The screenshot displays the 'Register' screen of a web application. At the top, a dark blue navigation bar contains the 'WIDGET WORLD' logo on the left and several menu items: 'FEATURES', 'PRICING', 'HOW TO USE', 'ROADMAP', 'EXPLORE', 'NEW ACCOUNT', and 'PUBLISH WIDGET'. The main content area is a dark blue gradient. Centered on this background is a white registration form with a purple-to-white gradient header that reads 'Register'. The form includes three input fields: 'Username' with the value 'hammad', 'Email Address' with 'hammadshdev@gmail.com', and 'Password' with masked characters. Below these fields is a purple 'Register' button.

Login Screen

The login screen of the web app is secure, user-friendly, responsive, and provides a seamless and efficient user authentication experience.



The screenshot displays the 'Login Form' screen of the web application. It features the same dark blue navigation bar and background as the registration screen. Centered is a white login form with a purple-to-white gradient header that reads 'Login Form'. The form contains two input fields: 'Email Address' with 'hammadshdev@gmail.com' and 'Password' with masked characters. Below the password field is a checked checkbox labeled 'Remember me'. A purple 'Login' button is positioned below the checkbox. At the bottom of the form, there is a link that says 'Not a member? Signup now'.

Publishing Widget Screen

The widget publication screen provides a streamlined, user-friendly, and secure interface, facilitating seamless submission and management of new widgets across all devices.

The 'Create Widget' form is a central modal on a dark blue background. It contains several input fields: 'Widget Name' with the value 'Test Widget', 'Code' with HTML snippets, a 'Category' dropdown set to 'List View', and an 'Image URL' field with a file upload button and a sample image. A 'Publish Your Widget' button is at the bottom.

Field	Value
Widget Name	Test Widget
Code	<pre><h1>Test Widget</h1> <p>This widget is for testing</p></pre>
Category	List View
Image URL	medium-shot-kl...23-2150385341.jpg

User dashboard

The user dashboard provides an intuitive interface where users can view, update, and delete their widgets, offering streamlined management and control over their projects.

The 'My Widgets' dashboard displays a table with columns for Widget Name, Status, Category, Image, Date, and Actions. It lists three widgets: 'Navbar13 Hammad' (Approved), 'Talha Widget' (Rejected), and 'Test Widget' (Approved).

Widget Name	Status	Category	Image	Date	Actions
Navbar13 Hammad	Approved	nav bar		8/12/2024	<button>Update</button> <button>Delete</button>
Talha Widget	Rejected	mixed		8/12/2024	<button>Update</button> <button>Delete</button>
Test Widget	Approved	list view		8/13/2024	<button>Update</button> <button>Delete</button>

Updating Widget Screen

Users can modify existing Flutter widgets through the platform. They can make changes to the widget's code, design, or functionality, and then resubmit for review and potential re-publication.

The screenshot shows the 'Update Widget' interface on the Widget World platform. The form is centered on a dark blue background. It contains the following fields and elements:

- Widget Name:** A text input field containing 'Navbar13 Hammad'.
- Code:** A code editor area showing Dart code for a Flutter widget. The code includes imports, a class definition for 'Navbar7' extending 'StatelessWidget', and an '@override' method.
- Category:** A dropdown menu currently showing 'Navigation Bar'.
- Image URL:** A section with a 'Choose File' button and the text 'No file chosen'.
- Update Widget:** A large purple button at the bottom of the form.

The top navigation bar of the platform is visible, featuring links for FEATURES, PRICING, HOW TO USE, ROADMAP, EXPLORE, LOGOUT, and PUBLISH WIDGET.

Admin Dashboard

Admin dashboard manages Flutter widget submissions, enabling approval or rejection based on quality and compliance criteria.

FEATURESPRICINGHOW TO USEROADMAPEXPLORELOGOUTPENDING WIDGET APPROVALS

Pending Widget Approvals

ttes111
Category: test
Owner: obaid1234
Date: 8/13/2024
Status: Rejected

ApproveReject

test123
Category: TEST
Owner: obaid1234
Date: 8/13/2024
Status: pending

ApproveReject

TestHSh
Category: Test
Owner: obaid1234
Date: 8/13/2024
Status: Pending

ApproveReject

Test HASHmi
Category: TEST
Owner: obaid1234
Date: 8/13/2024
Status: Rejected

ApproveReject

Test hashmi
Category: Test
Owner: obaid1234
Date: 8/13/2024
Status: Rejected

ApproveReject

Talha Widget
Category: mixed
Owner: hammad
Date: 8/13/2024
Status: Rejected

ApproveReject

test
Category: mixed
Owner: ADMIN
Date: 8/13/2024
Status: Pending

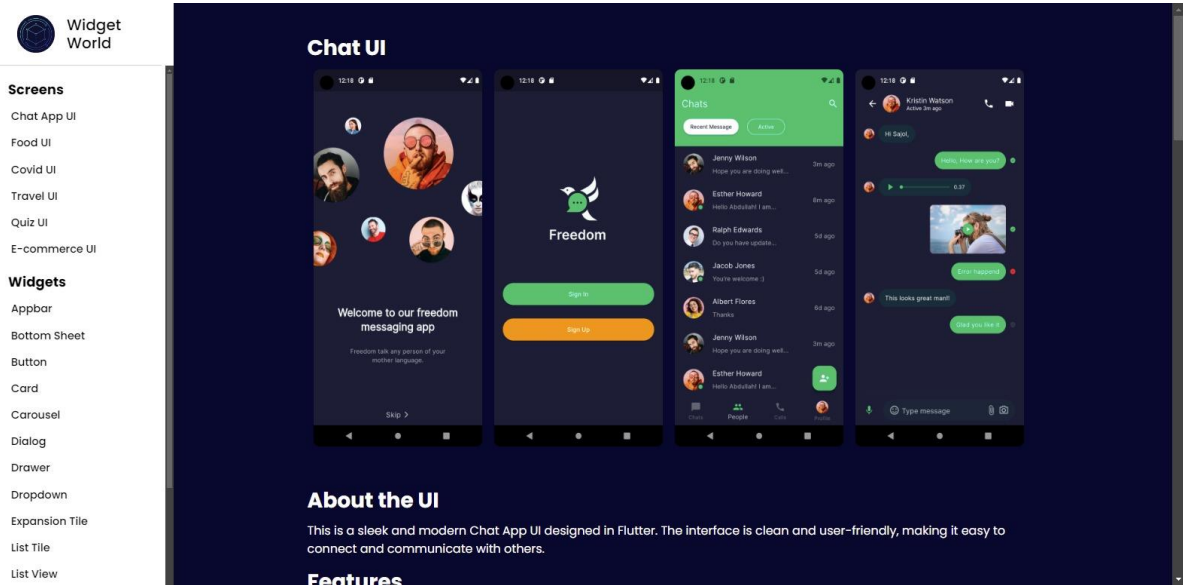
ApproveReject

dsafd
Category: search-bar
Owner: ADMIN
Date: 8/13/2024
Status: Pending

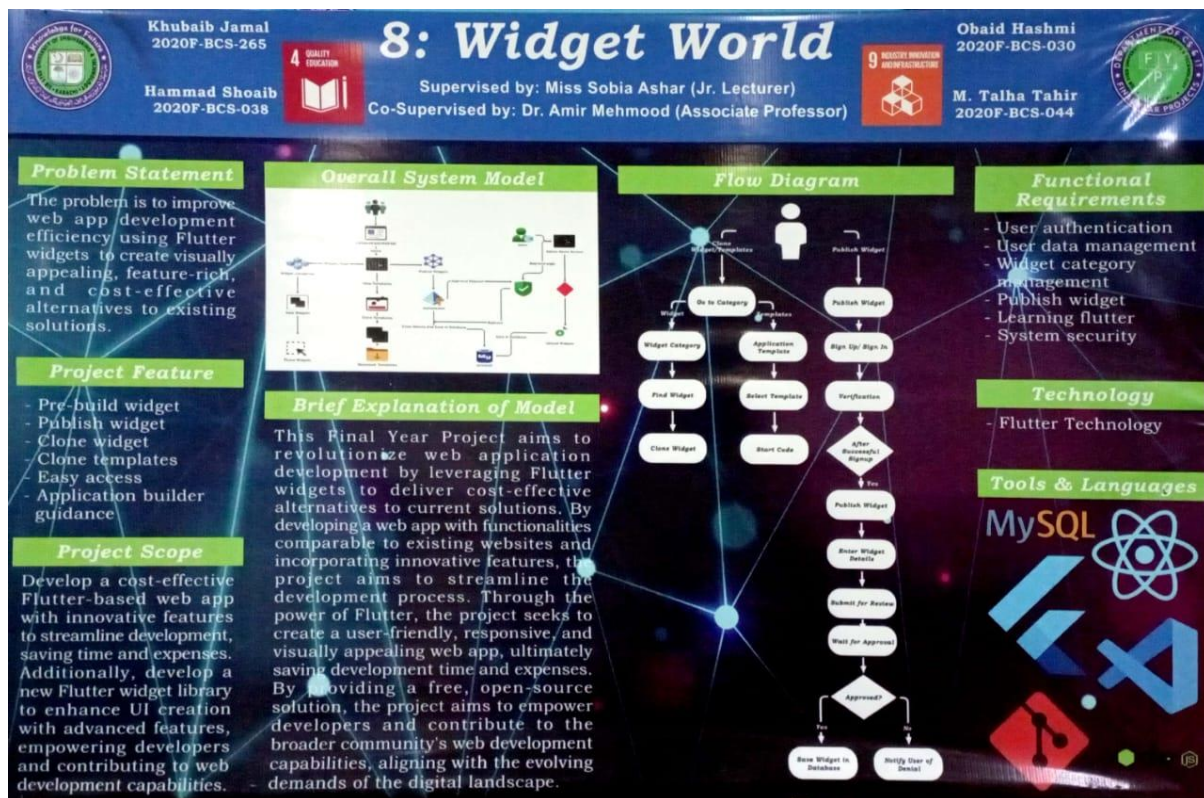
ApproveReject

Template

The template section allows users to quickly copy and implement pre-built, responsive templates into their projects, simplifying the development process across all devices.

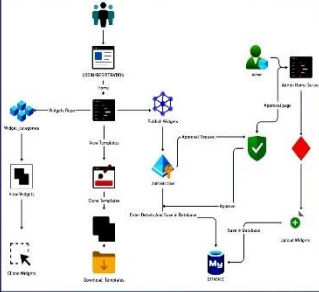


Appendix 2: Wall Poster



Appendix 3: Brochure

Overall System Model



Tools & Languages





Technology

- Flutter Technology



Project Members

Obaid Hashmi
2020F-BCS-030

Hammad Shoaib
2020F-BCS-038

M. Talha Tahir
2020F-BCS-044

Khubaib Jamal
2020F-BCS-265

Supervised by:
Miss Sobia Ashar

Co-Supervised by:
Dr. Amir Mehmood



4 QUALITY EDUCATION



9 INDUSTRY, INNOVATION AND INFRASTRUCTURE



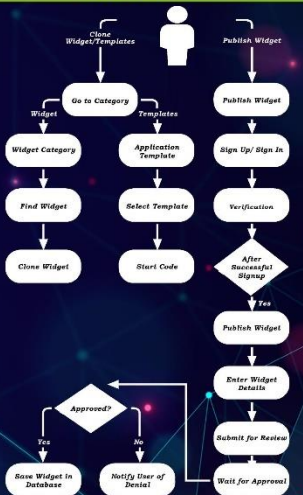

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
Main University Road,
Karachi-75300, Pakistan
Call : +92 21 3498 8000
Website : www.ssuet.edu.pk



Brief Explanation of Model

This Final Year Project aims to revolutionize web application development by leveraging Flutter widgets to deliver cost-effective alternatives to current solutions. By developing a web app with functionalities comparable to existing websites and incorporating innovative features, the project aims to streamline the development process. Through the power of Flutter, the project seeks to create a user-friendly, responsive, and visually appealing web app, ultimately saving development time and expenses. By providing a free, open-source solution, the project aims to empower developers and contribute to the broader community's web development capabilities, aligning with the evolving demands of the digital landscape.

Flow Diagram



Concept & Outline

The platform will offer a user-friendly interface for browsing, selecting, and integrating widgets into projects, promoting seamless code integration and customization. Users can modify widgets to fit project needs, with an intuitive navigation system enhancing the experience. Community engagement will allow contributions and feedback, while comprehensive documentation ensures usability. Version management ensures compatibility, while a new Flutter library expands features and simplifies UI creation, ultimately enhancing app engagement and providing developers a competitive edge.

Project Scope

Develop a cost-effective Flutter-based web app with innovative features to streamline development, saving time and expenses. Additionally, develop a new Flutter widget library to enhance UI creation with advanced features, empowering developers and contributing to web development capabilities.



4 QUALITY EDUCATION



9 INDUSTRY, INNOVATION AND INFRASTRUCTURE

FLUTTER: EMPOWERING DEVELOPERS, SHAPING THE FUTURE OF APPS

Concept & Outline

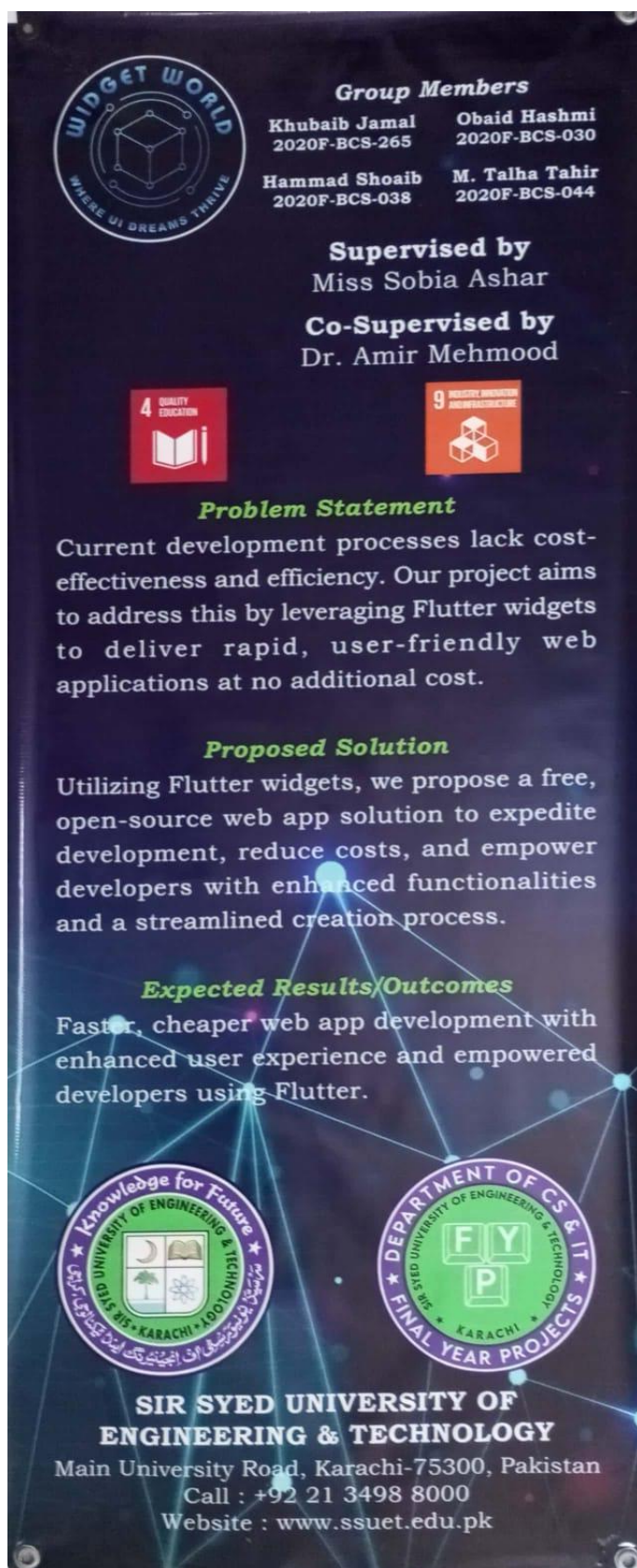
- Pre-build widget
- Publish widget
- Clone widget
- Clone templates
- Easy access
- Application builder guidance



Concept & Outline

This project seeks to revolutionize web development by utilizing Flutter to expedite and economize the process. Its primary objective is to provide developers with a free, open-source solution for creating responsive, user-friendly web apps. Specific goals include streamlining development through reusable Flutter widgets, reducing both time and costs compared to current solutions, and enhancing user experience with innovative features. Ultimately, this project aims to benefit the broader web development community by offering a potent, accessible toolset that caters to the evolving demands of the digital realm.

Appendix 4: Standee



Appendix 5: Industry Collaboration Letter



Getweys. Ltd
37C Mezzanine Floor 13th Commercial Street
DHA Phase II Extension
Karachi, Pakistan.

August 12, 2024

Sir Syed University of Engineering and Technology,
Computer Science,
Karachi.

Dear Widget World Team,

We are delighted to express our sincere appreciation for the outstanding work your team has done in developing the "Widget World" project, which was created as a part of your final year project and tailored specifically for our client.

We are impressed with the project and love the innovative features that "Widget World" offers. The ability to effortlessly copy and paste widgets into applications, coupled with the option for users to publish their own widgets, is a game-changer in the Flutter development space. This tool not only simplifies the development process but also fosters a collaborative environment where developers can share and benefit from each other's contributions.

Your dedication, creativity, and technical expertise have resulted in a product that exceeds expectations. We expressed high praise for the quality, functionality, and user-friendliness of the solution, noting that it has already started making a positive impact on their development workflow.

We are confident that the skills and knowledge you have demonstrated throughout this project will be invaluable in your future careers. Your work on "Widget World" stands as a testament to your capabilities, and we are proud to have collaborated with such a talented team.

Thank you once again for your remarkable contribution. We wish you continued success and look forward to seeing the positive influence you will undoubtedly have in the tech industry.

Warm Regards,



Muhammad Talha
(CEO)

1 | Page

37C Mezzanine Floor 13th Commercial Street DHA Phase II Extension Karachi, Pakistan | +923000184999
info@getweys.com

PAGE FOR REMARKS / COMMENTS

PAGE FOR REMARKS / COMMENTS